GitHub MCP Tool — Code Generation Prompt for Google Antigravity

Context

I am building **Milo**, an autonomous AI program coordinator built on a FastAPI + LangGraph + AWS Bedrock stack. Milo uses a set of MCP (Model Context Protocol) tool functions to interact with external systems (email, calendar, storage, work items, etc.).

I need you to build a **GitHub MCP server** that exposes the following tools to Milo's LangGraph agent runtime.

Stack & Conventions

- **Backend:** Python 3.11+, FastAPI, AWS Lambda (containerized), deployed via AWS CDK
- **Auth:** GitHub Personal Access Token (PAT) or GitHub App — stored in AWS Secrets Manager under key `milo/github/token`
- **MCP Pattern:** Each tool is a POST endpoint at `/services/mcp/github/<tool_name>` returning `{ "result": <data> }` or `{ "error": <message> }`
- **Existing MCP examples:** Follow the same request/response shape as `/services/mcp/nylas/email_read` and `/services/mcp/nylas/calendar_read` already in the codebase
- **Tenant isolation:** All calls must be scoped to the authenticated tenant's configured GitHub org/repo stored in the tenant settings table

---

Tools to Implement

1. `github__read_issues`

Endpoint: `POST /services/mcp/github/read_issues`

Input:

```
{
  "repo": "owner/repo",
  "state": "open | closed | all",
  "labels": ["optional", "label", "filters"],
  "limit": 25
}
```

Output: Array of `{ id, number, title, body, state, labels, assignees, created_at, updated_at, url }`

---

2. `github__read_pull_requests`

Endpoint: `POST /services/mcp/github/read_pull_requests`

Input:

```
{
  "repo": "owner/repo",
  "state": "open | closed | merged",
  "limit": 25
}
```

Output: Array of `{ id, number, title, body, state, head_branch, base_branch, author, reviewers, ci_status, created_at, updated_at, url }`

---

3. `github__read_ci_status`

Endpoint: `POST /services/mcp/github/read_ci_status`

Input:

```
{
  "repo": "owner/repo",
  "branch": "main"
}
```

Output: `{ branch, status: "success | failure | pending | unknown", workflow_runs: [{ name, status, conclusion, url, updated_at }] }`

---

4. `github__create_issue`

Endpoint: `POST /services/mcp/github/create_issue`

Input:

```
{
  "repo": "owner/repo",
  "title": "Issue title",
  "body": "Markdown body",
  "labels": ["optional"],
  "assignees": ["optional-github-username"]
}
```

Output: `{ id, number, url, title, state }`

---

5. `github__create_branch`

Endpoint: `POST /services/mcp/github/create_branch`

Input:

```
{
  "repo": "owner/repo",
  "branch_name": "phase/3-agent-runtime",
```

```
  "from_branch": "main"
}
```

Output: `{ branch_name, sha, url }`

---

6. `github__post_comment`

Endpoint: `POST /services/mcp/github/post_comment`

Input:

```
{
  "repo": "owner/repo",
  "issue_or_pr_number": 42,
  "body": "Markdown comment body"
}
```

Output: `{ comment_id, url, created_at }`

---

7. `github__read_commits`

Endpoint: `POST /services/mcp/github/read_commits`

Input:

```
{
  "repo": "owner/repo",
  "branch": "main",
  "limit": 20
}
```

Output: Array of `{ sha, message, author, timestamp, url, files_changed }`

Key Requirements

1. **Secrets Manager integration** — token fetched at cold start, cached in memory, refreshed on 401
2. **Error handling** — all GitHub API errors must return structured `{ "error": "<message>", "github_status": <code> }` — never raise unhandled exceptions
3. **Rate limit awareness** — check `X-RateLimit-Remaining` header; if < 100, include `"rate_limit_warning": true` in response
4. **CDK construct** — provide a `GitHubMcpConstruct` in CDK that wires the Lambda, API Gateway route, and Secrets Manager IAM policy
5. **Unit tests** — pytest with mocked `httpx` calls for all 7 tools, targeting 90%+ coverage
6. **LangGraph tool definitions** — provide the Python `@tool` decorated function stubs (with docstrings and typed inputs) ready to drop into Milo's tool catalog

---

Auto-Issue Creation Hook (Priority Feature)

When Milo calls `developer__handoff`, the system should **automatically call** `github__create_issue` with:

- Title = handoff title
- Body = description + acceptance criteria formatted as a GitHub checklist
- Label = `milo-handoff`
- Assignee = repo default assignee from tenant settings

This closes the loop between Milo's program tracker and the actual GitHub backlog.

Deliverables

1. `services/mcp/github/` — FastAPI router with all 7 endpoints
2. `cdk/constructs/github_mcp_construct.py` — CDK construct
3. `tests/mcp/test_github_mcp.py` — pytest suite
4. `agent/tools/github_tools.py` — LangGraph `@tool` stubs
5. `docs/mcp/github_mcp.md` — brief integration doc

Notes

- Use `httpx` (async) for all GitHub API calls — consistent with existing MCP services
- GitHub REST API v3 base URL: `https://api.github.com`
- Accept header: `application/vnd.github+json`
- All endpoints must be async FastAPI routes